

Table of content

Preamble..... 2

Isolation tests.....2

 Memory.....2

 Array.....2

 Math.....2

 String.....3

 Output.....3

 Screen.....3

 Keyboard.....4

 Sys.....4

Global test..... 5

Preamble

Jack code written in 2025 and translated into english in 2026.

Isolation tests

Memory

The proposed `Memory.jack` source file implements a first-fit strategy for the improved memory allocation algorithm (see § 12.1.3). Here is how to test it :

1. Recompile Jack source files :

```
c:\(...)\nand2tetris\projects\12>call ..\..\tools\JackCompiler.bat Perso\MemoryTest  
compiling "c:\(...)\nand2tetris\projects\12\Perso\MemoryTest"
```

2. Open the VM Emulator : `call ..\..\tools\VMEmulator.bat`

File / Load Script / Select `c:\(...)\nand2tetris\projects\12\Perso\MemoryTest\MemoryTest.tst`

Select Animate / No animation to get a fast answer.

3. Run the test (`>>`), choose *Yes* for the Confirmation Message, click *OK* when execution has finished.

4. Check that *End of script - Comparison ended successfully* appears at the bottom of the screen.

As a complementary test, the same procedure can be applied to `Perso\MemoryTest\MemoryDiag`.

However, I do not remember having done a step-by-step debugging in the VM Emulator to fully test the `Memory.alloc()` and `Memory.deAlloc()` methods.

Array

Same 4 steps as for the *Memory* isolation test.

Math

Again, same 4 steps as for the *Memory* isolation test with one more verification. If step 4 has succeeded, check the memory :

- Go to the 8000 memory location using the binocular tool.
- Check that `RAM[8000 → 8013]` contains the values described in `MathTest\Main.jack`.

String

Here, the test procedure changes a bit. We have the same 3 first steps as the *Memory* isolation test but the fourth is different. Here, we have to check that the screen output is identical to the screen capture in the documentation, that is :

```
new,appendChar: abcde
setInt: 12345
setInt: -32767
length: 5
charAt[2]: 99
setCharAt(2,'-'): ab-de
eraseLastChar: ab-d
intValue: 456
intValue: -32123
backSpace: 129
doubleQuote: 34
newLine: 128
```

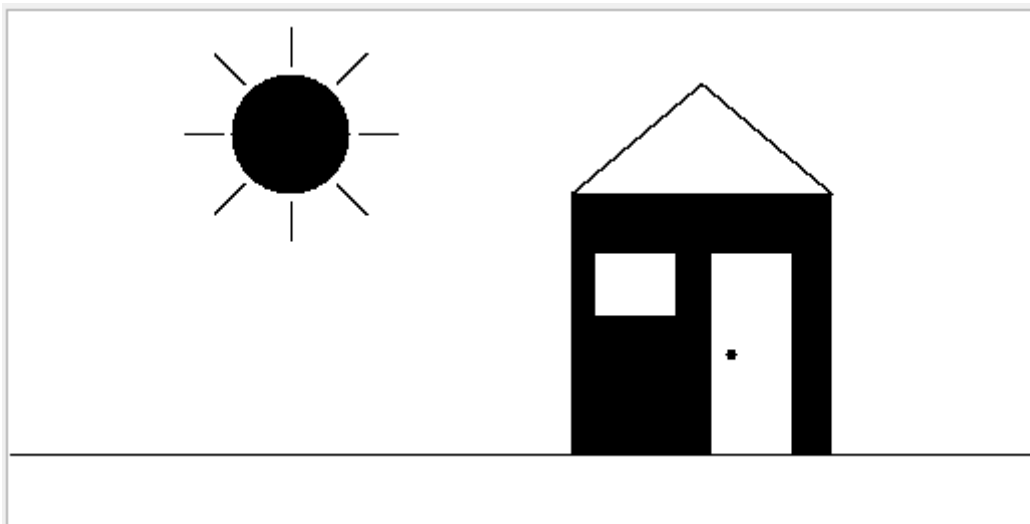
Output

Same procedure as for the previous test. The result has to be identical to this :

```
A
0123456789
ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz
!#$%&'()*+,-./:;<=>?@[ ]^_`{|}~"
-12346789
C                                     D
```

Screen

Same test again. The result has to be identical to this :



Keyboard

Same test but some typing to do this time. Expected result :

```
keyPressed test:
Please press the 'Page Down' key
ok
readChar test:
(Verify that the pressed character is echoed to the screen)
Please press the number '3': 3
ok
readLine test:
(Verify echo and usage of 'backspace')
Please type 'JACK' and press enter: JACK
ok
readInt test:
(Verify echo and usage of 'backspace')
Please type '-32123' and press enter: -32123
ok
Test completed successfully
```

Sys

Similar test except we have to check that ~ 2 seconds have passed between the moment we press a key and the moment the *Time is up* message appears on the screen :

```
Wait test:
Press any key. After 2 seconds, another message will be printed:
Time is up. Make sure that 2 seconds elapsed.
```

Global test

The global test should be run once the 8 isolation tests are ok. This final test is done with the *Pong* game. The word *game* being quite exaggerated here since the result is not playable at all (especially for what is supposed to be an arcade game !). Indeed, the Jack code contains nearly none optimization and is therefore very slow. The main bottleneck is obviously `screen.jack`. However, some interesting hints are given on the nand2tetris forum. See for example [here](#) and particularly what is said about the `drawHorizontalLine()` function.

